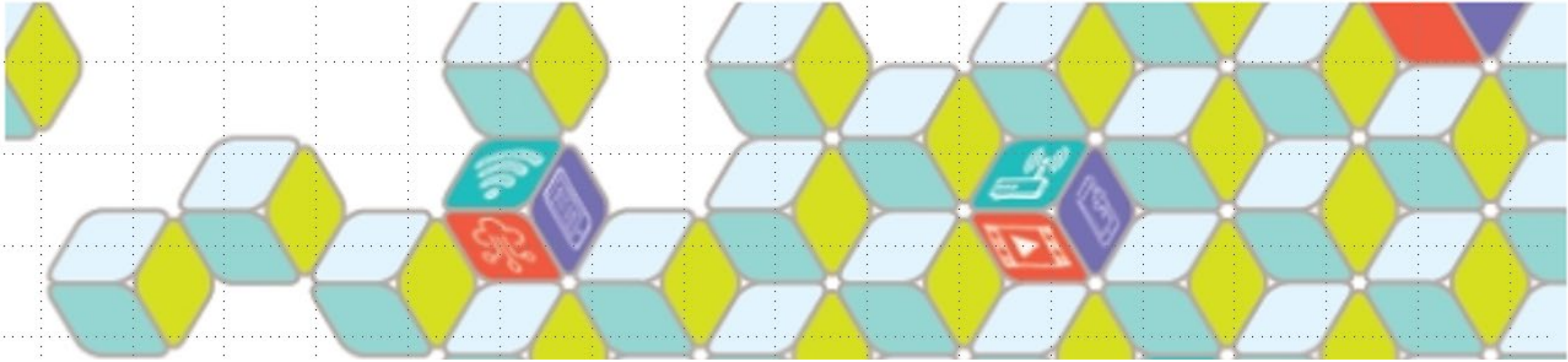


Programming Method
(using C, C++, for Games)

Prof. Tae Soo Yun
Dept. of Game, International College, Dongseo University

DSU Dongseo University 동서대학교

Ch04_Functions and Program Structure

The Goal of the ch. 04-Functions and Program Structure

Functions are a bunch of code for performing a certain function.

Functions break large computing tasks into smaller ones, and enable people to build on what others have done instead of starting over from scratch. Appropriate functions hide details of operation from parts of the program that don't need to know about them, thus clarifying the whole, and easing the pain of making changes.

Chapter 4

- Section 4.1 Basics of Functions
- Section 4.2 Functions Retruning Non-integers
- Section 4.3 External Variables
- Section 4.4 Scope Rules
- Section 4.5 Header Files
- Section 4.6 Static Variables
- Section 4.7 Register Variables
- Section 4.8 Block Structure
- Section 4.9 Initialization
- Section 4.10 Recursion
- Section 4.11 The C Preprocessor

4.1 Basics of Functions

❖ Function Definition

✓ has the form

```
return-type function-name(argument declarations){
        declarations and statements
} or

dummy() {}  // minimal function type
```

✓ A program is just a set of definitions of variables and functions.

✓ If the return type is omitted, int is assumed.

✓ Communication between the functions is by arguments and values returned by the functions, and through external variables.

✓ Return statement is the mechanism for returning a value from the called function to its caller : **return expression** ;

Example of function

- write a program to print each line of its input that contains a particular “pattern” or string of characters.
  - ✓ Search for the pattern of letters “ ould” in the set of lines

```
Ah Love! could you and I with Fate conspire
To grasp this sorry Scheme of Things entire,
Would not we shatter it to bits -- and then
Re-mould it nearer to the Heart’s Desire!
```

  - ✓ Will produce the output

```
Ah Love! could you and I with Fate conspire
Would not we shatter it to bits -- and then
Re-mould it nearer to the Heart’s Desire!
```

  - ✓ The job falls neatly into three pieces:

```
while (there's another line)
    if (the line contains the pattern)
        print it
```

example

❖ download link

```
#include <stdio.h>
#define MAXLINE 1000    /* maximum input line length */

int getline(char line[], int max);
int strindex(char source[], char searchfor[]);

char pattern[] = "ould";    /* pattern to search for */

/* find all lines matching pattern */
main()
{
    char line[MAXLINE];
    int found = 0;

    while (getline(line, MAXLINE) > 0)
        if (strindex(line, pattern) >= 0) {
            printf("%s", line);
            found++;
        }
    return found;
}
```

```
/* getline:  get line into s, return length */
int getline(char s[], int lim)
{
    int c, i;

    i = 0;
    while (--lim > 0 && (c=getchar()) != EOF && c != '\n')
        s[i++] = c;
    if (c == '\n')
        s[i++] = c;
    s[i] = '\0';
    return i;
}

/* strindex:  return index of t in s, -1 if none */
int strindex(char s[], char t[])
{
    int i, j, k;

    for (i = 0; s[i] != '\0'; i++) {
        for (j=i, k=0; t[k]!='\0' && s[j]==t[k]; j++, k++)
            ;
        if (k > 0 && t[k] == '\0')
            return i;
    }
    return -1;
}
```

Exercise 4-1

❖ exercise 4-1:

- ✓ Write the function strrindex(s,t), which returns the position of the rightmost occurrence of t in s, or -1 if there is none.
- ✓ Think about how to fix or modify the function to meet ex4-1.
- ✓ Ex) s[] = “abcdedfabcde”, t[] =“abc” , rightmost index = 7

```
int strindex(char s[], char t[]) {
    int i, j, k;

    for (i = 0; s[i] != '\0'; i++) {
        for (j = i, k = 0; t[k] != '\0' && s[j] == t[k]; j++, k++)
            ;
        if (k > 0 && t[k] == '\0')
            return i;
    }
    return -1;
}
```

4.3 External Variables

- A C program consists of a set of external object, which are either variables or functions.
- external variables are defined outside of any functions, & are thus potentially available to many functions.
- external var. &functions are called by the same name.
- ext. var. provide an alternative to function arguments and return values for communicating data between functions.
- Advantage of external var.
  - ✓ Are more convenient and efficient than long argument list.
  - ✓ Have their greater scope and lifetime.
  - ✓ Automatic (Internal) var.s are come into existence when the function is entered, and disappear when it is left.
  - ✓ External var.s are permanent.
  - ✓ If two functions share some data, it is often most convenient

  if the shared data is kept in external variables rather than passed in and out via arguments.
- Examine this issue further with an example. (calculator)

Postfix notation example

❖ write a program to calculate the equation with polish notation (postfix) instead of infix

✓ Infix notation (1−2) * (4 +5) is entered as 1 2 - 4 5 + *

✓ The structure of the program

```
while (next operator or operand is not end-of-file indicator)
    if (number)
        push it
    else if (operator)
        pop operands
        do operation
        push result
    else if (newline)
        pop and print top of stack
    else
        error
```

```
#includes
#defines

function declarations for main

main() { ... }

external variables for push and pop

void push(double f) { ... }
double pop(void) { ... }

int getop(char s[]) { ... }

routines called by getop
```

❖Download link

```
#include <stdio.h>
#include <stdlib.h>    /* for atof() */

#define MAXOP    100    /* max size of operand or operator */
#define NUMBER   '0'    /* signal that a number was found */

int getop(char []);
void push(double);
double pop(void);

/* reverse Polish calculator */
main()
{
    int type;
    double op2;
    char s[MAXOP];

    while ((type = getop(s)) != EOF) {
        switch (type) {
        case NUMBER:
            push(atof(s));
            break;
        case '+':
            push(pop() + pop());
            break;
        case '*':
            push(pop() * pop());
            break;
        case '-':
            op2 = pop();
            push(pop() - op2);
            break;
        case '/':
            op2 = pop();
            if (op2 != 0.0)
                push(pop() / op2);
            else
                printf("error: zero divisor\n");
            break;
        case '\n':
            printf("\t%.8g\n", pop());
            break;
        default:
            printf("error: unknown command %s\n", s);
            break;
        }
    }
    return 0;
}
```

```c
#define MAXVAL   100    /* maximum depth of val stack */

int sp = 0;             /* next free stack position */
double val[MAXVAL];     /* value stack */

/* push:  push f onto value stack */
void push(double f)
{
    if (sp < MAXVAL)
        val[sp++] = f;
    else
        printf("error: stack full, can't push %g\n", f);
}



/* pop:  pop and return top value from stack */
double pop(void)
{
    if (sp > 0)
        return val[--sp];
    else {
        printf("error: stack empty\n");
        return 0.0;
    }
}
```

```c
#include <ctype.h>

int getch(void);
void ungetch(int);

/* getop:  get next operator or numeric operand */
int getop(char s[])
{
    int i, c;

    while ((s[0] = c = getch()) == ' ' || c == '\t')
        ;
    s[1] = '\0';
    if (!isdigit(c) && c != '.')
        return c;     /* not a number */
    i = 0;
    if (isdigit(c))   /* collect integer part */
        while (isdigit(s[++i] = c = getch()))
            ;
    if (c == '.')     /* collect fraction part */
        while (isdigit(s[++i] = c = getch()))
            ;
    s[i] = '\0';
    if (c != EOF)
        ungetch(c);
    return NUMBER;
}
```

4.4 Scope Rules

- The Scope of a name is the part of the program within which the name can be used.
  - ✓ That is, how far does the variable's range extend?
- write a program that divides calculate program into several files.
  - ✓ main(), sp, val, push(), pop() are defined in one file.
  - ✓ sp, val[] are defined and initialized in another file.
  - ✓ We should declare data type with extern keyword.

In file 1:

```
extern int sp;
extern double val[];

void push(double f) { … }
double pop(void) { … }
```

In file 2:

```
int sp = 0;
extern double val[MAXVAL];
```

4.5 header files

❖ consider dividing the calculator program into several files.

- ✓ main.c
- ✓ stack.c
- ✓ getop.c
- ✓ getch.c
- ✓ clac.h is header file that contains the prototype definition and other define #
  - specify " "

main.c:

```
#include <stdio.h>
#include <stdlib.h>
#include "calc.h"
#define MAXOP 100
main() {
    ...
}
```

getop.c:

```
#include <stdio.h>
#include <ctype.h>
#include "calc.h"
getop() {
    ...
}
```

stack.c:

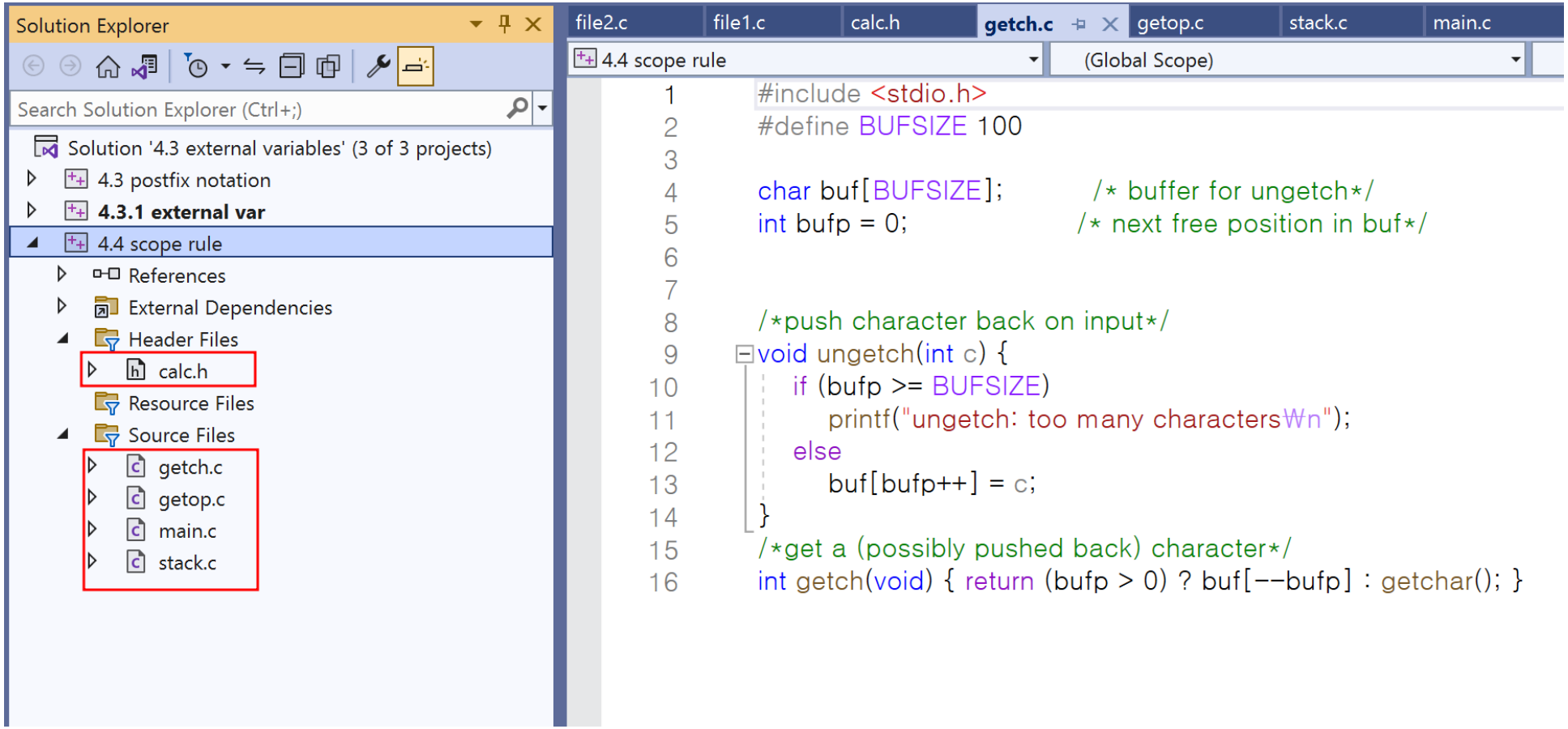
```
#include <stdio.h>
#include "calc.h"
#define MAXVAL 100
int sp = 0;
double val[MAXVAL];
void push(double) {
    ...
}
double pop(void) {
    ...
}
```

getch.c:

```
#include <stdio.h>
#define BUFSIZE 100
char buf[BUFSIZE];
int bufp = 0;
int getch(void) {
    ...
}
void ungetch(int) {
    ...
}
```

calc.h:

```
#define NUMBER '0'
void push(double);
double pop(void);
int getop(char []);
int getch(void);
void ungetch(int);
```


Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution '4.3 external variables' (3 of 3 projects)
4.3 postfix notation
4.3.1 external var
4.4 scope rule
References
External Dependencies
Header Files
calc.h
Resource Files
Source Files
getch.c
getop.c
main.c
stack.c
file2.c
file1.c
calc.h
getch.c
getop.c
stack.c
main.c
4.4 scope rule
(Global Scope)
1 #include <stdio.h>
2 #define BUFSIZE 100
3
4 char buf[BUFSIZE]; /* buffer for ungetch*/
5 int bufp = 0; /* next free position in buf*/
6
7
8 /*push character back on input*/
9 void ungetch(int c) {
10 if (bufp >= BUFSIZE)
11 printf("ungetch: too many characters₩n");
12 else
13 buf[bufp++] = c;
14 }
15 /*get a (possibly pushed back) character*/
16 int getch(void) { return (bufp > 0) ? buf[--bufp] : getchar(); }

4.11 The C Preprocessor

- in previous chapter, two usage of preprocessor, #include , #define
  - ✓ #include inform the compiler to include the file
  - ✓ #define, to replace a token by an arbitrary sequence of character
- provides conditional compilation and macros with arguments

- 4.11.1 File inclusion
  - ✓ #include is used for compounding of external variables and declaration of complicated data types, and can be only one definition and command.
  - ✓ It guarantees that all the source files have the same definition and variable definition.

- 4.11.2 Macro Substitution
  - ✓ #define name replacement text : replace the occurrences of the token(name) with replacement text
  - ✓ #define forever for(;;) : /*infinite loop*/

❖ Macro substitution

✓ #define max(A, B) ((A) >(B) ? (A) : (B)),

- x = max(p+q, r+s) **will be replaced by** x= ((p+q) > (r+s) ? (p+q) : (r+s));

❖4.11.3 Conditional inclusion

✓ #undef : resolve the macro definition

✓ #if, #else, #endif : compile according to the condition

✓ #ifdef, #endif : compile if macro is defined

✓ #ifndef, #endif : compile if macro is not defined

✓ #line : print the line number

❖ the examples of preprocessor directives : #ifdef, #endif

✓ Syntax

```
#ifdef MACRO
        statements
#endif
```

```
#define DEBUG
int average(int x, int y){
#ifdef DEBUG
    printf("x=%d, y=%d\n", x, y);
#endif
}
```

```
//#define DEBUG
int average(int x, int y){
#ifdef DEBUG
    printf("x=%d, y=%d\n", x, y);
#endif
}
```

❖ #if, #else, #endif

```
#if (DEBUF == 1)
printf(" ", var…);
#endif
```

```
#if 0
void test(){
    /*   …    */
    sub();
}
#endif
```

```
#define NATION 1
#if NATION == 1
printf("안녕하세요");
#elif NATION == 2
printf("Hello world");
#else
printf("other language");
#endif
```

❖#ifndef, #endif, for preventing header file conflict.

```
/*  EXAMPLE.H - Example header file  */
#if !defined( EXAMPLE_H )
#define EXAMPLE_H

class Example
{
    //...
};

#endif // !defined( EXAMPLE_H )
```

exercise

Exercise 4-3. Given the basic framework, it's straightforward to extend the calculator. Add the modulus (`%`) operator and provisions for negative numbers. □

Exercise 4-4. Add commands to print the top element of the stack without popping, to duplicate it, and to swap the top two elements. Add a command to clear the stack. □

Exercise 4-5. Add access to library functions like `sin`, `exp`, and `pow`. See `<math.h>` in Appendix B, Section 4. □

Exercise 4-6. Add commands for handling variables. (It's easy to provide twenty-six variables with single-letter names.) Add a variable for the most recently printed value. □

Exercise 4-7. Write a routine `ungets(s)` that will push back an entire string onto the input. Should `ungets` know about `buf` and `bufp`, or should it just use `ungetch`? □

Exercise 4-8. Suppose that there will never be more than one character of pushback. Modify `getch` and `ungetch` accordingly. □

Exercise 4-9. Our `getch` and `ungetch` do not handle a pushed-back `EOF` correctly. Decide what their properties ought to be if an `EOF` is pushed back, then implement your design. □

Exercise 4-10. An alternate organization uses `getline` to read an entire input line; this makes `getch` and `ungetch` unnecessary. Revise the calculator to use this approach. □